IntelliSDLC

Automated UI Testing with Selenium WebDriver

Test report with screenshots

Name: Mihir Bhati

Test run: 20 October 2026, 20:11-20:12

Result: 13 of 13 test cases passed

1. Objective

The objective of this test is to verify, using Selenium WebDriver, that a user can sign in to the IntelliSDLC web application and that every main page of the application opens and renders correctly. A screenshot of each page is captured automatically by Selenium as evidence.

2. Test Environment

Automation tool	Selenium WebDriver 4.51.0 (Python bindings)
Browser	Google Chrome 155.0.8059.85
Driver	ChromeDriver 155.0.8059.84 (downloaded automatically by Selenium Manager)
Language	Python 3
Application frontend	React + Vite, http://localhost:5173
Application backend	FastAPI (Uvicorn), http://127.0.0.1:8000
Operating system	Windows 11
Window size	1440 x 900
Test account	Admin demo account (admin@intellisdlc.dev)

3. Scope of Testing

This is a smoke test of the user interface. Selenium covered:

- Opening the login page.
- Signing in with the Admin demo account.
- Navigating to all 11 pages that are available after sign-in and confirming each page loads with the correct heading.
- Capturing a screenshot of every page.

Not covered by this Selenium run: creating, editing or deleting records (requirements, bugs, sprints and so on), form validation, and role-based permissions. Those are tested separately by the project's backend test suite.

4. How the Test Works

The test script is tools/selenium_screenshots.py. Its steps:

- **Launch:** webdriver.Chrome() opens a real Chrome window controlled by Selenium.
- **Open:** driver.get() opens the login page; WebDriverWait waits for the email field to appear.
- **Sign in:** find_element() locates the 'Admin' demo-account card and click() signs in.
- **Navigate:** for each page, driver.get() opens the route, the script waits for the page content and reads the h1 heading.
- **Capture:** execute_script() adds a footer naming Selenium, the browser, the URL and the time, then save_screenshot() saves the PNG.
- **Log:** every step is written with a timestamp to selenium_run_log.txt.

Command used to run the test:

```
backend\.venv\Scripts\python.exe tools\selenium_screenshots.py
```

5. Test Cases and Results

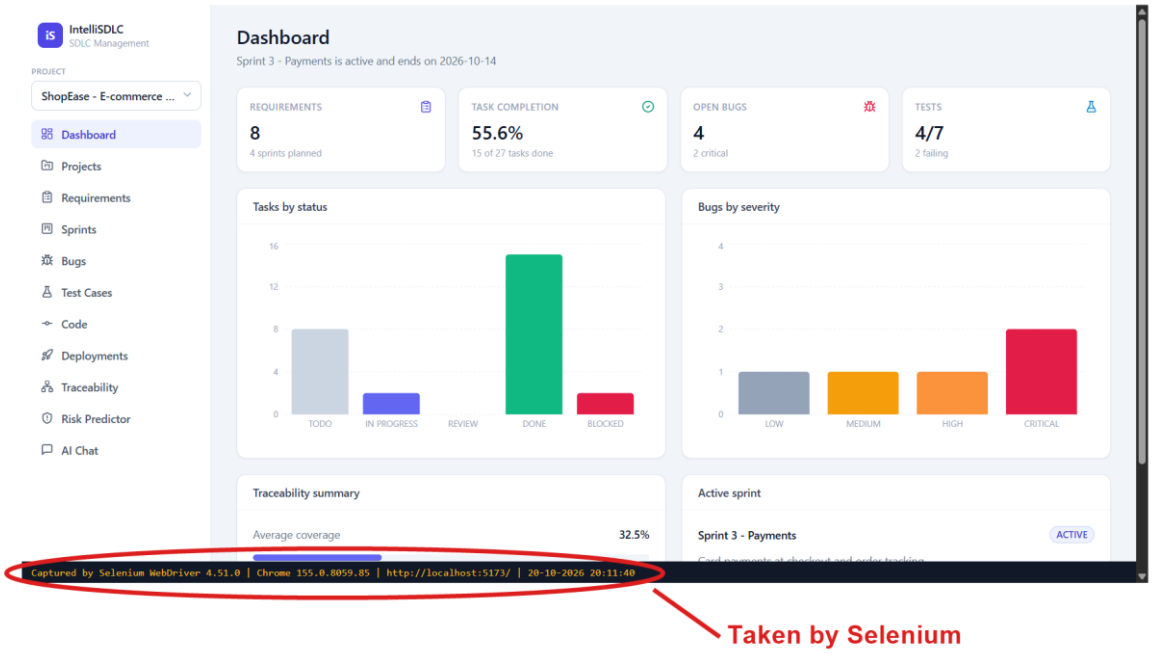
TC-01	Open login page	Open /login	Email & password fields visible	PASS
TC-02	Sign in (Admin demo)	Click 'Admin' demo account	Redirected to dashboard	PASS
TC-03	Dashboard page loads	Open /	Page loads with heading 'Dashboard'	PASS
TC-04	Projects page loads	Open /projects	Page loads with heading 'Projects'	PASS
TC-05	Requirements page loads	Open /requirements	Page loads with heading 'Requirements'	PASS
TC-06	Sprints page loads	Open /sprints	Page loads with heading 'Sprints'	PASS
TC-07	Bugs page loads	Open /bugs	Page loads with heading 'Bugs'	PASS
TC-08	Test Cases page loads	Open /test-cases	Page loads with heading 'Test Cases'	PASS
TC-09	Code page loads	Open /code	Page loads with heading 'Code'	PASS
TC-10	Deployments page loads	Open /deployments	Page loads with heading 'Deployments'	PASS
TC-11	Traceability page loads	Open /traceability	Page loads with heading 'Traceability Matrix'	PASS
TC-12	Risk Predictor page loads	Open /risk	Page loads with heading 'Sprint Risk Predictor'	PASS
TC-13	AI Chat page loads	Open /chat	Page loads with heading 'AI Project Agent'	PASS

Summary: 13 test cases executed, 13 passed, 0 failed.

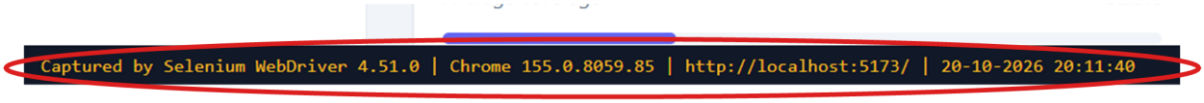
6. Screenshots Captured by Selenium

6.1 How to identify a Selenium screenshot

Before saving each screenshot, the script injects a footer bar into the page using driver.execute_script(). The bar shows the Selenium WebDriver version, the browser version, the page URL and the exact time of capture. It is marked with a red circle in the example below (Dashboard page):



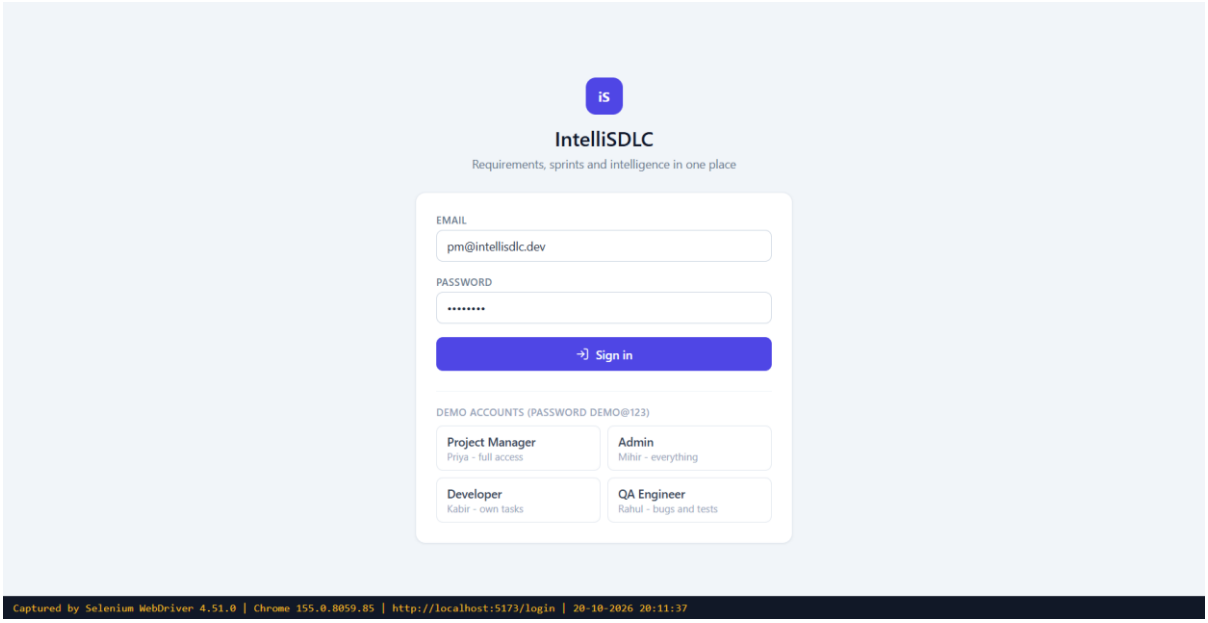
Zoomed view of the marked footer:



Every screenshot in section 6.2 carries the same footer.

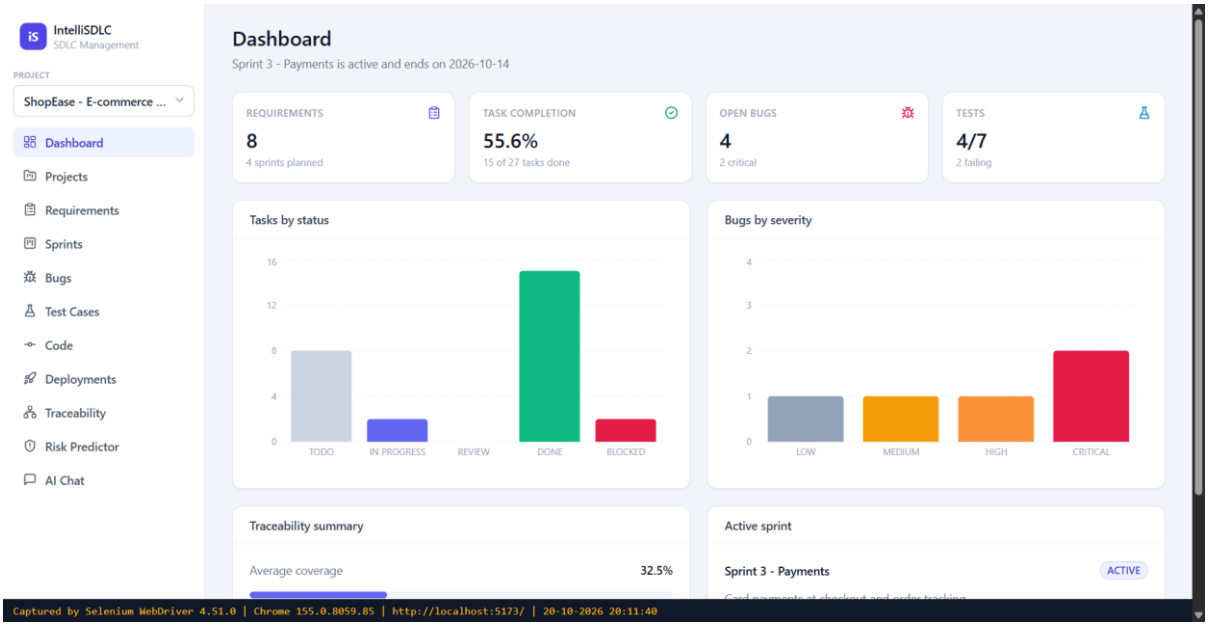
6.2 Screenshots of all pages

Figure 1: Login (/login)



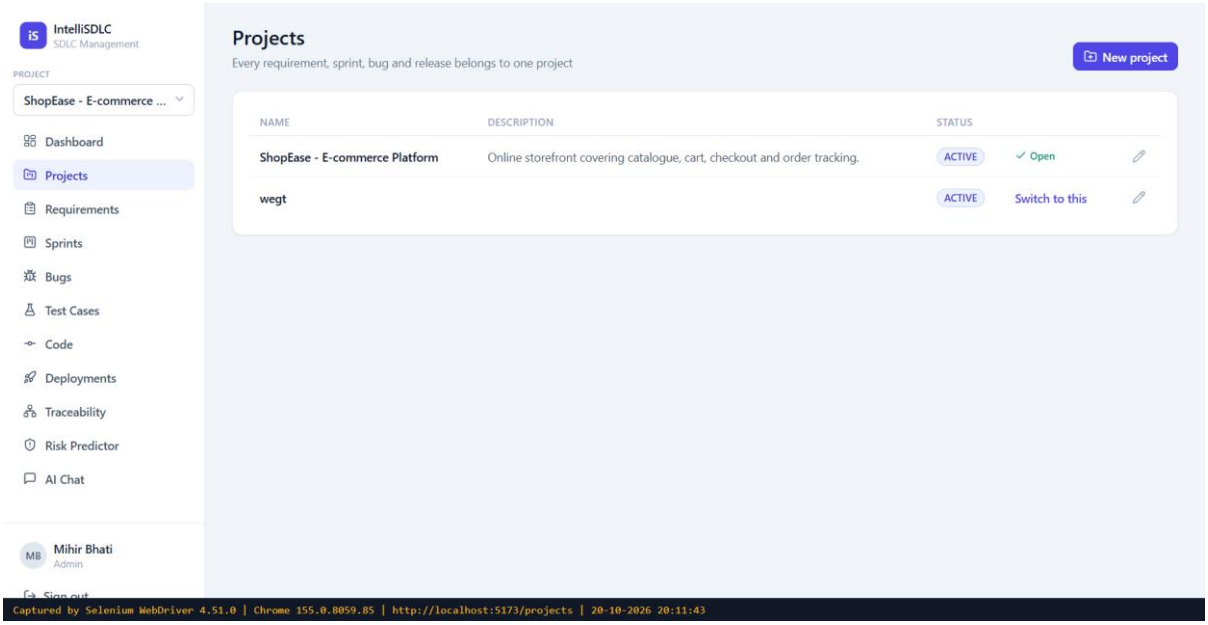
Login page. The user signs in with email and password, or picks one of the demo accounts (Project Manager, Admin, Developer, QA). Selenium opened this page first.

Figure 2: Dashboard (/)



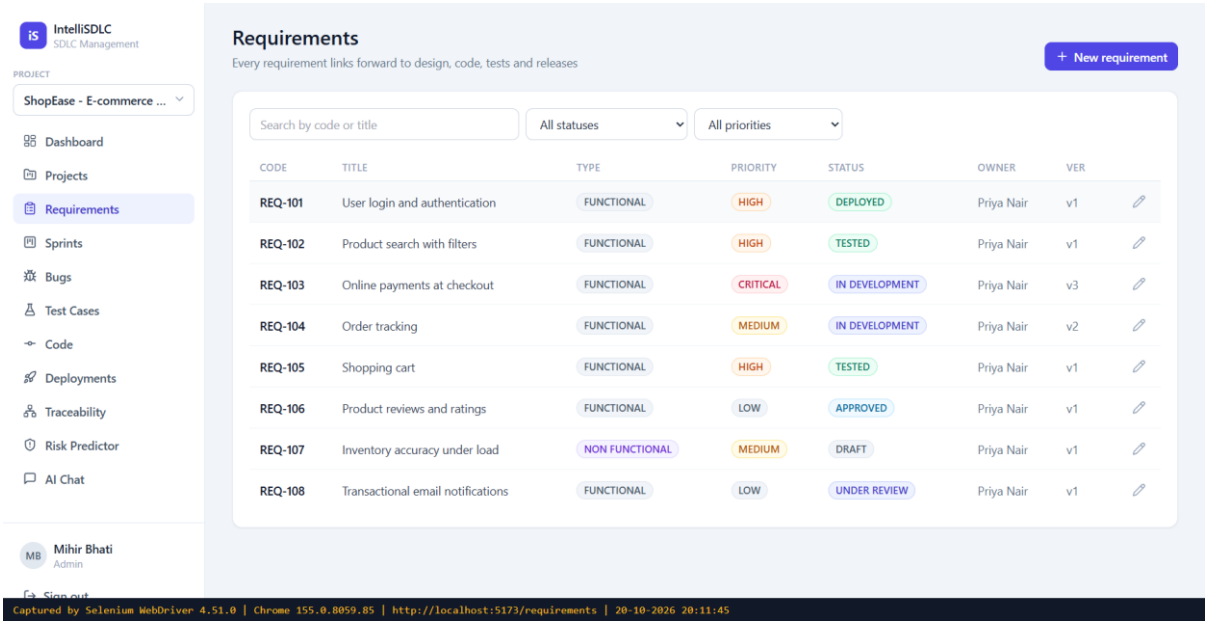
Dashboard after login. It shows the project summary: requirements, task completion, open bugs and tests, with charts for tasks by status and bugs by severity.

Figure 3: Projects (/projects)



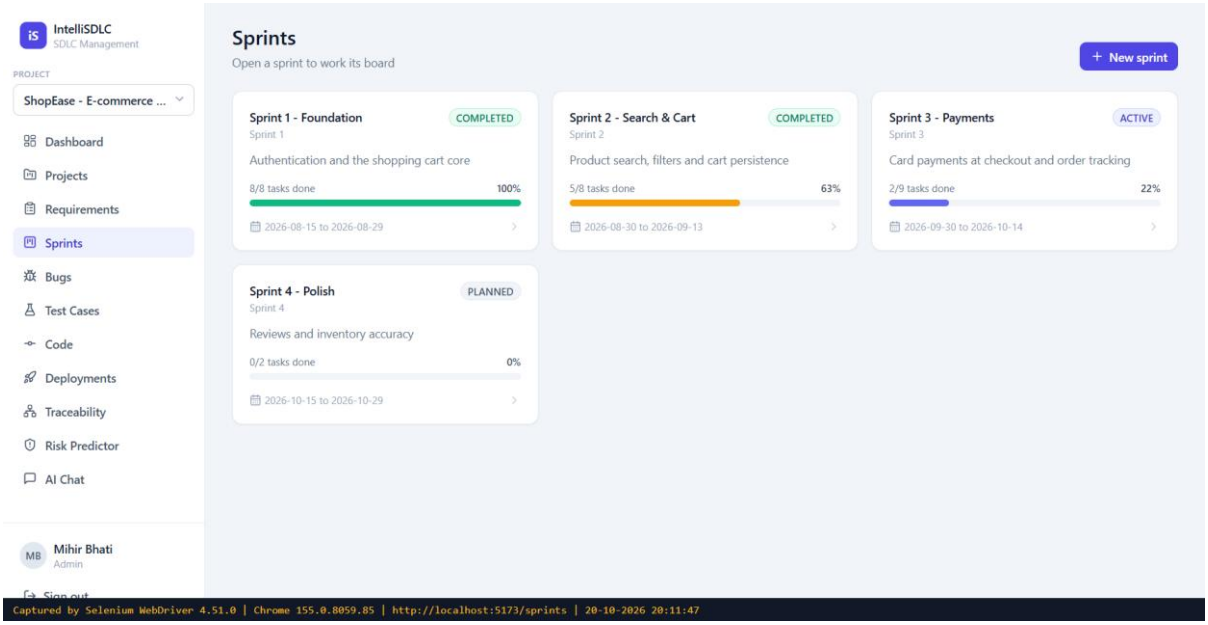
Projects page. Lists all projects; every requirement, sprint, bug and release belongs to one project.

Figure 4: Requirements (/requirements)



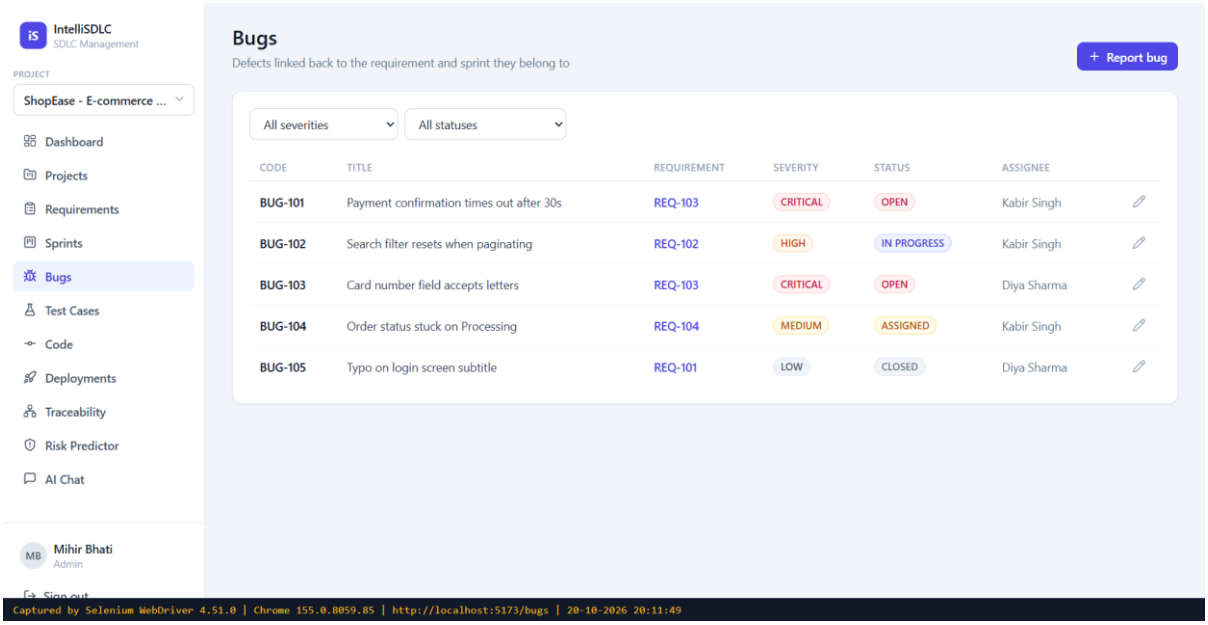
Requirements page. Lists the project's requirements; each one is linked to its design, code, tests and releases.

Figure 5: Sprints (/sprints)



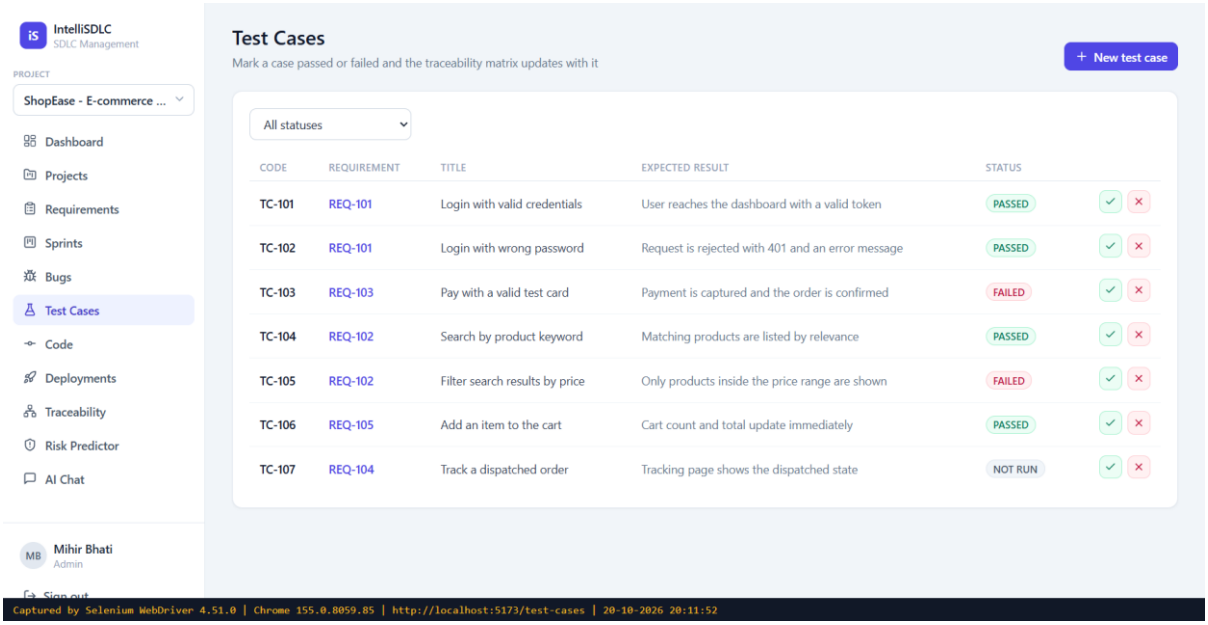
Sprints page. Shows all sprints of the project; opening a sprint shows its task board.

Figure 6: Bugs (/bugs)



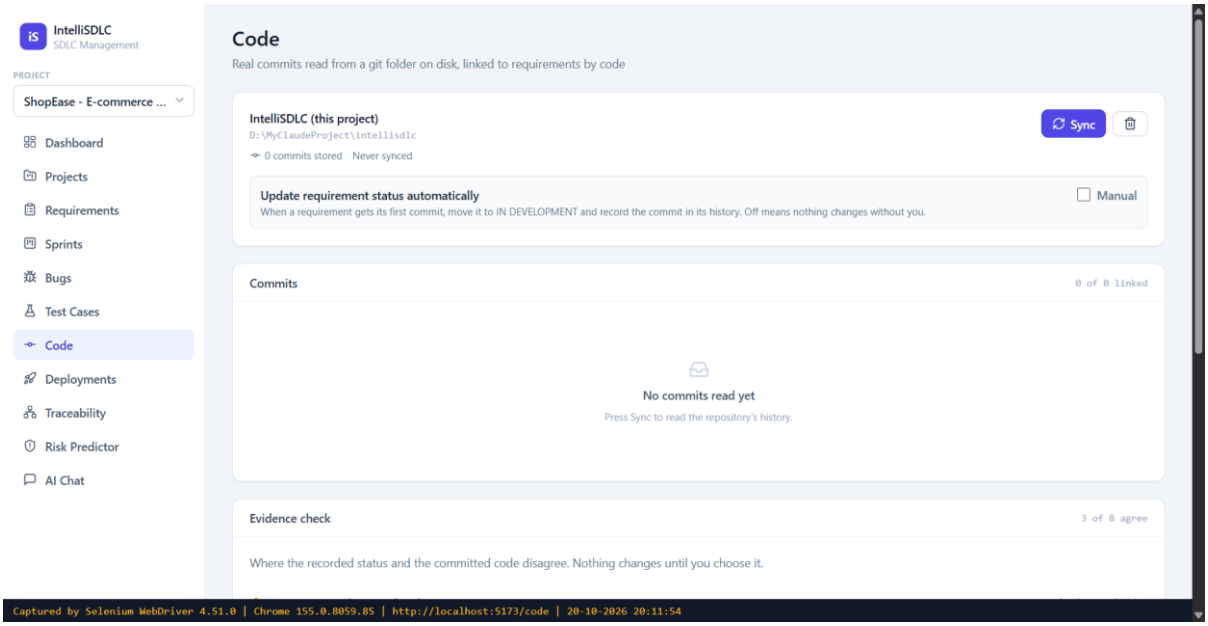
Bugs page. Lists defects with their severity and status, each linked to its requirement and sprint.

Figure 7: Test Cases (/test-cases)



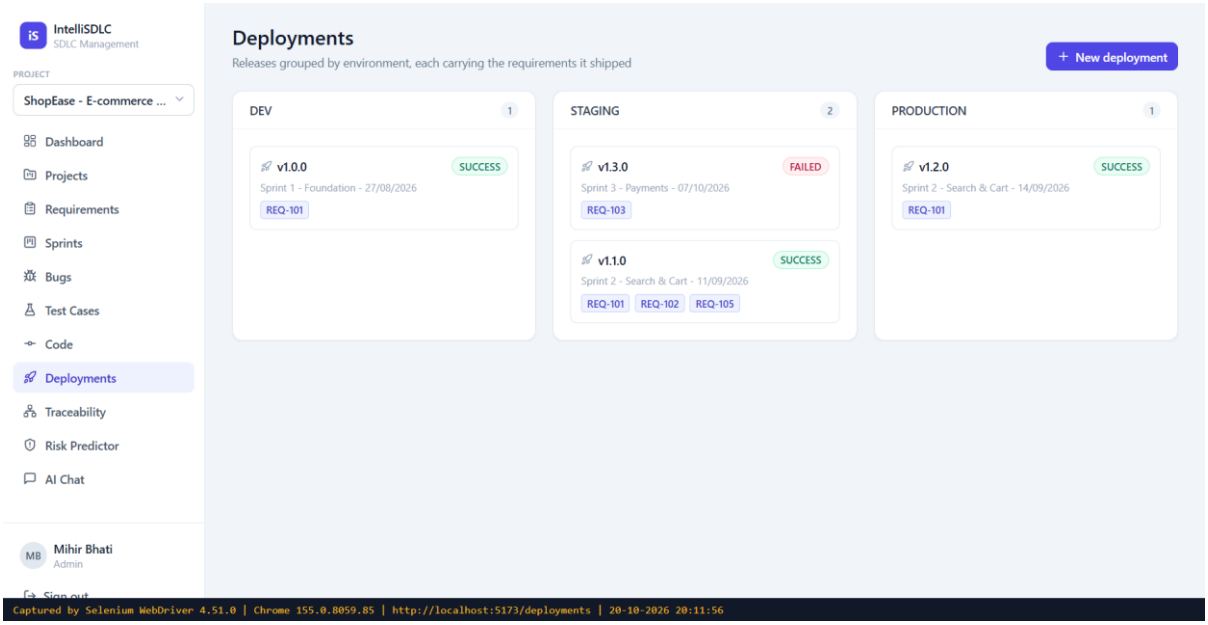
Test Cases page. Lists test cases; marking one passed or failed updates the traceability matrix.

Figure 8: Code (/code)



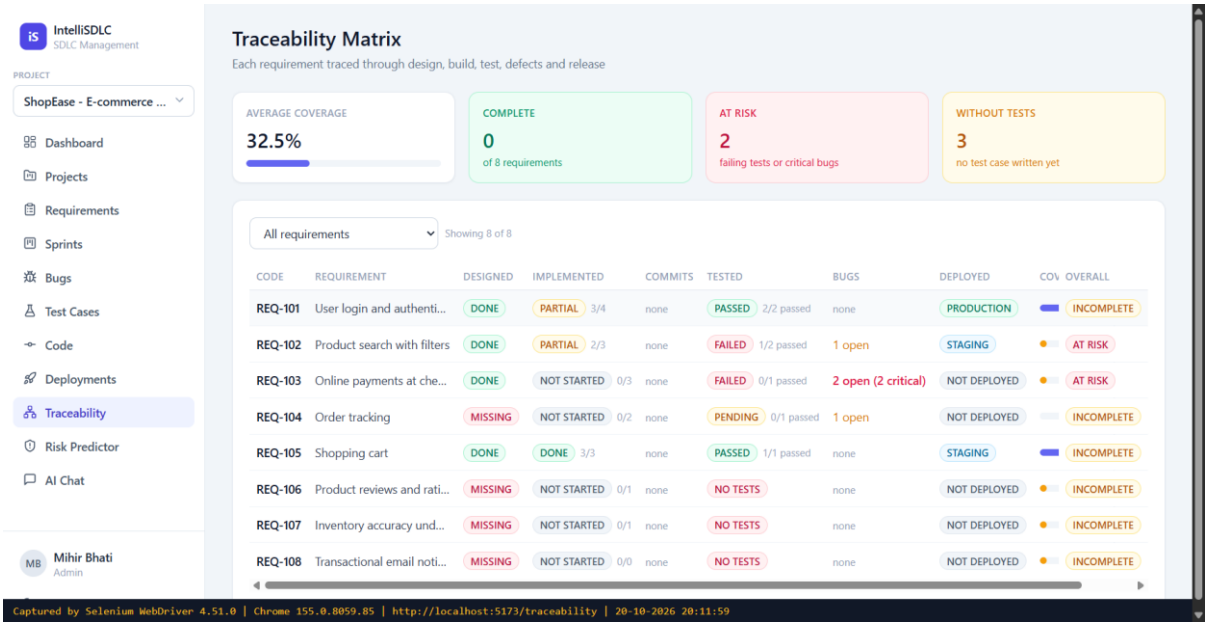
Code page. Shows git commits read from the project repository and links them to requirements.

Figure 9: Deployments (/deployments)



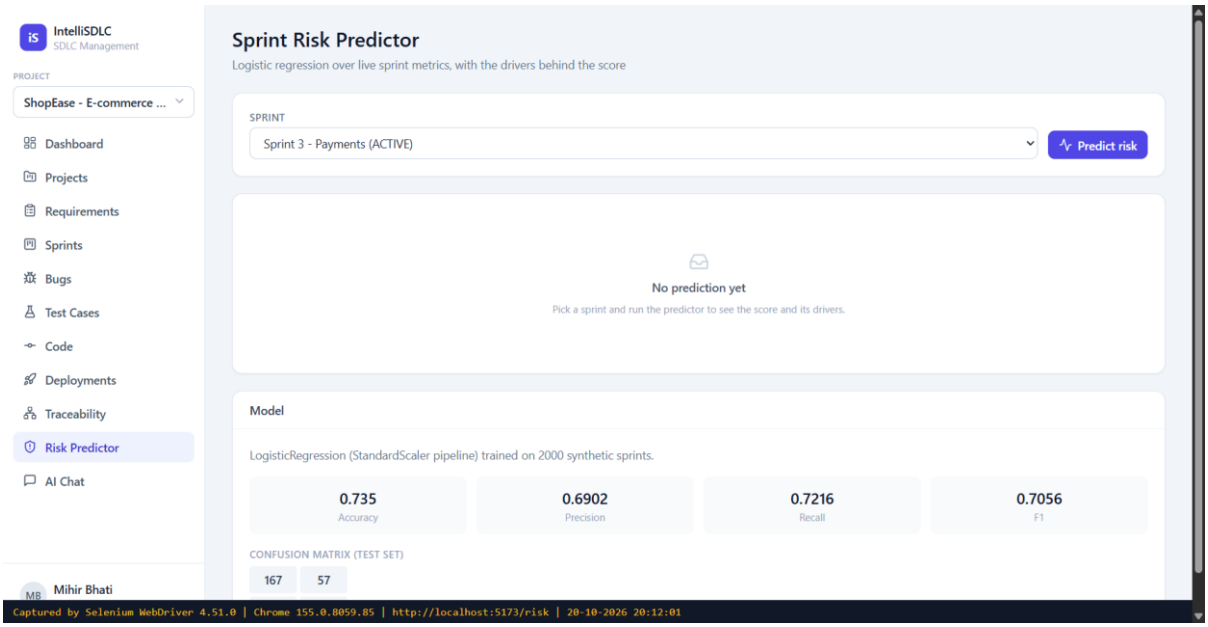
Deployments page. Shows releases grouped by environment, with the requirements each release shipped.

Figure 10: Traceability (/traceability)



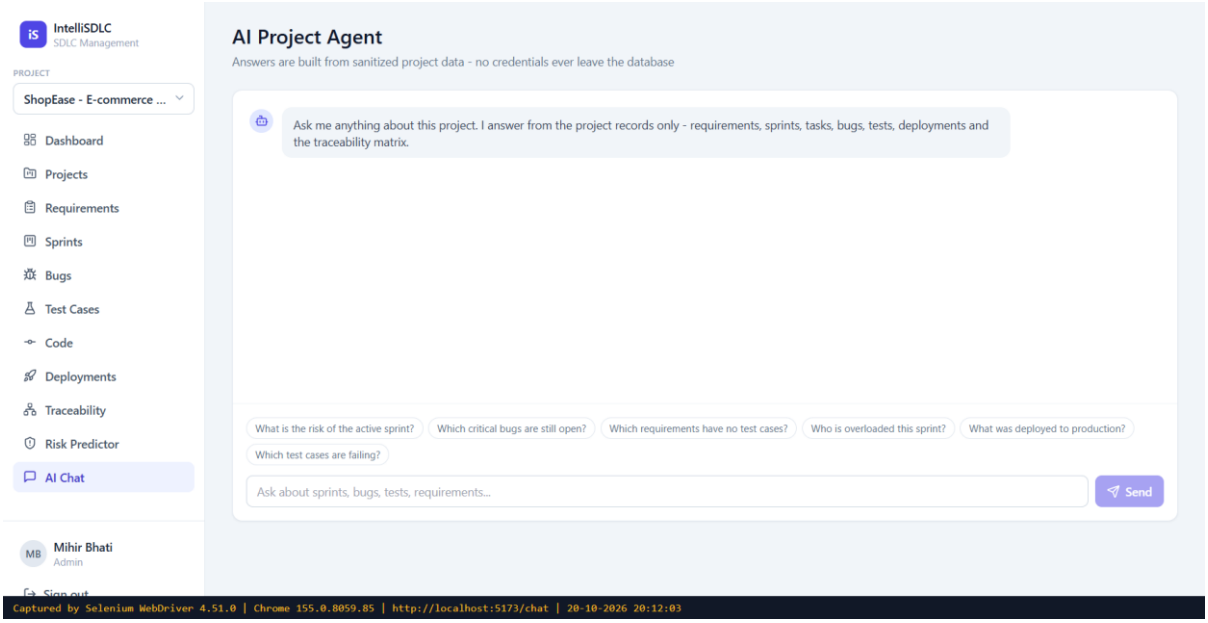
Traceability Matrix. Traces every requirement through design, build, test, defects and release.

Figure 11: Risk Predictor (/risk)



Sprint Risk Predictor. A machine-learning model (logistic regression) predicts the risk of the current sprint and shows what drives the score.

Figure 12: AI Chat (/chat)



AI Chat page. An assistant that answers questions about the project using the project's data.

7. Selenium Execution Log

This log was written by the script during the test run (selenium_run_log.txt):

```
[2026-10-20 20:11:36] Selenium 4.51.0 started Chrome 155.0.8059.85 (ChromeDriver 155.0.8059.84)
[2026-10-20 20:11:36] Opened http://localhost:5173/login - page title 'IntelliSDLC'
[2026-10-20 20:11:37] Screenshot saved: 01_login.png  (http://localhost:5173/login)
[2026-10-20 20:11:38] Clicked 'Admin' demo account
[2026-10-20 20:11:38] PASS: signed in, dashboard loaded
[2026-10-20 20:11:40] PASS: / loaded - heading 'Dashboard'
[2026-10-20 20:11:40] Screenshot saved: 02_dashboard.png  (http://localhost:5173/)
[2026-10-20 20:11:43] PASS: /projects loaded - heading 'Projects'
[2026-10-20 20:11:43] Screenshot saved: 03_projects.png  (http://localhost:5173/projects)
[2026-10-20 20:11:45] PASS: /requirements loaded - heading 'Requirements'
[2026-10-20 20:11:45] Screenshot saved: 04_requirements.png  (http://localhost:5173/requirements)
[2026-10-20 20:11:47] PASS: /sprints loaded - heading 'Sprints'
[2026-10-20 20:11:47] Screenshot saved: 05_sprints.png  (http://localhost:5173/sprints)
[2026-10-20 20:11:49] PASS: /bugs loaded - heading 'Bugs'
[2026-10-20 20:11:50] Screenshot saved: 06_bugs.png  (http://localhost:5173/bugs)
[2026-10-20 20:11:52] PASS: /test-cases loaded - heading 'Test Cases'
[2026-10-20 20:11:52] Screenshot saved: 07_test_cases.png  (http://localhost:5173/test-cases)
[2026-10-20 20:11:54] PASS: /code loaded - heading 'Code'
[2026-10-20 20:11:54] Screenshot saved: 08_code.png  (http://localhost:5173/code)
[2026-10-20 20:11:56] PASS: /deployments loaded - heading 'Deployments'
[2026-10-20 20:11:56] Screenshot saved: 09_deployments.png  (http://localhost:5173/deployments)
[2026-10-20 20:11:59] PASS: /traceability loaded - heading 'Traceability Matrix'
[2026-10-20 20:11:59] Screenshot saved: 10_traceability.png  (http://localhost:5173/traceability)
[2026-10-20 20:12:01] PASS: /risk loaded - heading 'Sprint Risk Predictor'
[2026-10-20 20:12:01] Screenshot saved: 11_risk.png  (http://localhost:5173/risk)
[2026-10-20 20:12:03] PASS: /chat loaded - heading 'AI Project Agent'
[2026-10-20 20:12:03] Screenshot saved: 12_ai_chat.png  (http://localhost:5173/chat)
[2026-10-20 20:12:03] Done. 12 screenshots in tools\selenium_screenshots
```

8. Selenium Test Script

Full source of tools/selenium_screenshots.py:

```
"""Selenium screenshot tour of IntelliSDLC.

Opens a visible Chrome window, signs in with a seeded demo account, visits
every page and saves a screenshot of each to tools/selenium_screenshots/.
Each screenshot carries a footer naming Selenium, the browser, the URL and
the time, and every step is written to selenium_run_log.txt.

Run with the backend (port 8000) and frontend (port 5173) already started:
    backend\\.venv\\Scripts\\python.exe tools\\selenium_screenshots.py
"""

import time
from datetime import datetime
from pathlib import Path

import selenium
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait

BASE_URL = "http://localhost:5173"
OUT_DIR = Path(__file__).parent / "selenium_screenshots"
LOG_FILE = OUT_DIR / "selenium_run_log.txt"

PAGES = [
    ("02_dashboard", "/"),
    ("03_projects", "/projects"),
    ("04_requirements", "/requirements"),
    ("05_sprints", "/sprints"),
    ("06_bugs", "/bugs"),
    ("07_test_cases", "/test-cases"),
    ("08_code", "/code"),
    ("09_deployments", "/deployments"),
    ("10_traceability", "/traceability"),
    ("11_risk", "/risk"),
    ("12_ai_chat", "/chat"),
]

# Footer injected into the page just before each screenshot
STAMP_JS = """
const old = document.getElementById('selenium-stamp');
if (old) old.remove();
const bar = document.createElement('div');
bar.id = 'selenium-stamp';
bar.textContent = arguments[0];
bar.style.cssText = 'position:fixed;left:0;right:0;bottom:0;z-index:2147483647;'
  + 'background:#111827;color:#fbbf24;font:13px Consolas,monospace;padding:6px 12px;';
document.body.appendChild(bar);
"""



def log(line: str) -> None:
    entry = f"[{datetime.now():%Y-%m-%d %H:%M:%S}] {line}"
    print(entry)
    with LOG_FILE.open("a", encoding="utf-8") as f:
        f.write(entry + "\n")



def take(driver, name: str, browser: str) -> None:
    stamp = (
        f"Captured by Selenium WebDriver {selenium.__version__} | {browser} | "
        f"{driver.current_url} | {datetime.now():%d-%m-%Y %H:%M:%S}"
    )
    driver.execute_script(STAMP_JS, stamp)
    driver.save_screenshot(str(OUT_DIR / f"{name}.png"))
```

```
    log(f"Screenshot saved: {name}.png  ({driver.current_url})")


def main() -> None:
    OUT_DIR.mkdir(exist_ok=True)
    LOG_FILE.write_text("", encoding="utf-8")

    options = webdriver.ChromeOptions()
    options.add_argument("--window-size=1440,900")
    driver = webdriver.Chrome(options=options)  # Selenium Manager fetches chromedriver
    wait = WebDriverWait(driver, 15)
    caps = driver.capabilities
    browser = f"{caps['browserName'].title()} {caps['browserVersion']}"
    log(f"Selenium {selenium.__version__} started {browser} "
        f"(ChromeDriver {caps['chrome']['chromedriverVersion'].split()[0]})")

    try:
        # Login page
        driver.get(f"{BASE_URL}/login")
        wait.until(EC.presence_of_element_located((By.CSS_SELECTOR, "input[type=email]")))
        log(f"Opened {driver.current_url} - page title '{driver.title}'")
        time.sleep(1)
        take(driver, "01_login", browser)

        # Sign in by clicking the "Admin" demo-account card on the login page
        admin_card = wait.until(
            EC.element_to_be_clickable((By.XPATH, "//*[normalize-space(text())='Admin']"))
        )
        admin_card.click()
        log("Clicked 'Admin' demo account")
        try:
            wait.until(EC.presence_of_element_located((By.TAG_NAME, "main")))
        except Exception:
            take(driver, "login_failed", browser)
            log("FAIL: sign-in did not reach the dashboard")
            raise
        log("PASS: signed in, dashboard loaded")

        # Every page after sign-in
        for name, path in PAGES:
            driver.get(f"{BASE_URL}{path}")
            wait.until(EC.presence_of_element_located((By.TAG_NAME, "main")))
            time.sleep(2)  # let charts and API data finish rendering
            heading = driver.find_elements(By.TAG_NAME, "h1")
            log(f"PASS: {path} loaded - heading '{heading[0].text if heading else '-'}'")
            take(driver, name, browser)

        log(f"Done. {len(PAGES) + 1} screenshots in {OUT_DIR}")
        time.sleep(2)
    finally:
        driver.quit()


if __name__ == "__main__":
    main()
```

9. Conclusion

Selenium WebDriver successfully automated Google Chrome to sign in to IntelliSDLC and open all 12 pages of the application. All 13 test cases passed, and a screenshot of every page was captured automatically as evidence. The test can be repeated at any time by running the script with the application started.